

CAD for VLSI PA2 : Partition

Student : 卓祐晟

Student ID : 111521051

Date : 2024/04/22

Course : CAD for VLSI Design

Supervisor : Professor 陳聿廣

Outline

- CAD for VLSI PA2 : Partition
 - Outline
 - 1. Assignment Requirement
 - 2. How to Compile and Execute
 - Test Result
 - 3. Pseudocode and Flow Chart
 - Pseudocode
 - Flow chart
 - 4. Program Details
 - Read input file to build netArray and cellArray
 - Neighbor partition data struct
 - Initial partition
 - Parameter in SA algorithm
 - Move to neighbor point
 - The probability of moving to the neighbor point
 - Cooling Schedule
 - Other modifications to the SA algorithm
 - 5. The Hardness I Encountered
 - How to set the parameters of SA algorithm is better?
 - How to improve SA by initial partition?
 - 6. My Suggestion

1. Assignment Requirement

Let $C = \{C_1, C_2, \dots, C_n\}$ be a set of cells and $N = \{N_1, N_2, \dots, N_m\}$ be a set of nets. Each net $N_i, i = 1, 2, \dots, m$, connects a subset of cells. The two-way min-cut partitioning problem is to partition the cell set into two groups A and B. The size of group A, $size(A)$, is the number of cells in group A. $size(B)$ is the number of cells in group B. The cost of a two-way partitioning is measured by the cut size, which is the number of nets having cells in both groups. The problem is defined as follows:

Input: A net-list for a circuit.

Output: Partition the circuit to two sub-circuits A and B so that the cut size of sub-circuits A and B is minimized.

Constraints:

- 1. $|size(A) - size(B)| \leq n/5$, where n is the number of cells in the circuit and n always > 5 .
- 2. The runtime of your program is limited to at most ****10 minutes**** per testcase.

2. How to Compile and Execute

Step1. Add a Makefile in folder PA2, choose GNU GCC Compiler in C++ 23 standard.

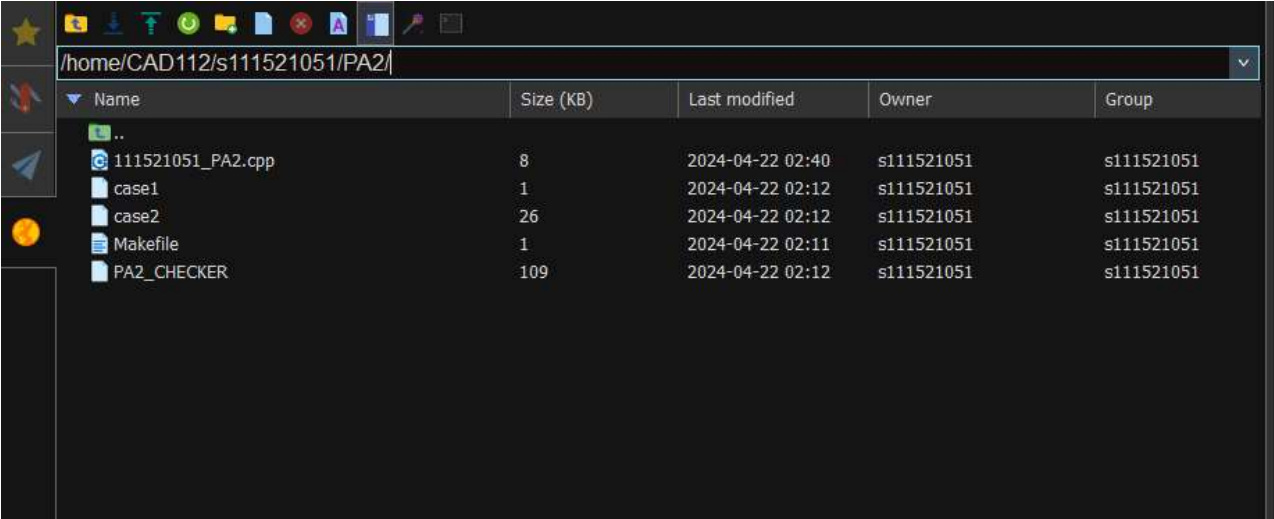
```
Makefile x
#PA2 cpp compiler
.PHONY: all run clean
all: 111521051_PA2.o
    @g++ -std=c++23 111521051_PA2.o -o exe

run:
    @./exe $(input) $(output)

clean:
    @rm *.o
    @rm exe

111521051_PA2.o: 111521051_PA2.cpp
    @g++ -std=c++23 -c 111521051_PA2.cpp
```

Step2. Upload the files 111521051_PA2.cpp, case1, case2, PA2_CHECKER.



Name	Size (KB)	Last modified	Owner	Group
111521051_PA2.cpp	8	2024-04-22 02:40	s111521051	s111521051
case1	1	2024-04-22 02:12	s111521051	s111521051
case2	26	2024-04-22 02:12	s111521051	s111521051
Makefile	1	2024-04-22 02:11	s111521051	s111521051
PA2_CHECKER	109	2024-04-22 02:12	s111521051	s111521051

Step3. Use make to compile and execute the program.

```
[s111521051@cad ~/PA2]$ ls
111521051_PA2.cpp  case1  case2  Makefile  PA2_CHECKER
[s111521051@cad ~/PA2]$ make all
[s111521051@cad ~/PA2]$ ls
111521051_PA2.cpp  111521051_PA2.o  case1  case2  exe  Makefile  PA2_CHECKER
[s111521051@cad ~/PA2]$ make run input=case1 output=case1.out
Excution Time: 0 s
[s111521051@cad ~/PA2]$ make run input=case2 output=case2.out
Excution Time: 12 s
```

Step4. Use the PA2_CHECKER executable file to check.

```
[s111521051@cad ~/PA2]$ ./PA2_CHECKER case1 case1.out
error count: 0
Well Done !!!

[s111521051@cad ~/PA2]$ ./PA2_CHECKER case2 case2.out
error count: 0
Well Done !!!

[s111521051@cad ~/PA2]$
```

Test Result

Test cutsize of case1 and case2

case1.out	
1	cut_size 3
2	A
3	c8
4	c3
5	c10
6	c7
7	B
8	c6
9	c2
10	c1
11	c4
12	c5
13	c9

case2.out	
1	cut_size 234
2	A
3	c50
4	c67
5	c51
6	c221
7	c284
8	c392
9	c429
10	c444
11	c413
12	c191
13	c2

3. Pseudocode and Flow Chart

Pseudocode

Read input file

Generate myCellArray and myNetArray

Initialize partition case initial_partition

best_partition = initial_partition

Set initial temperature temperature

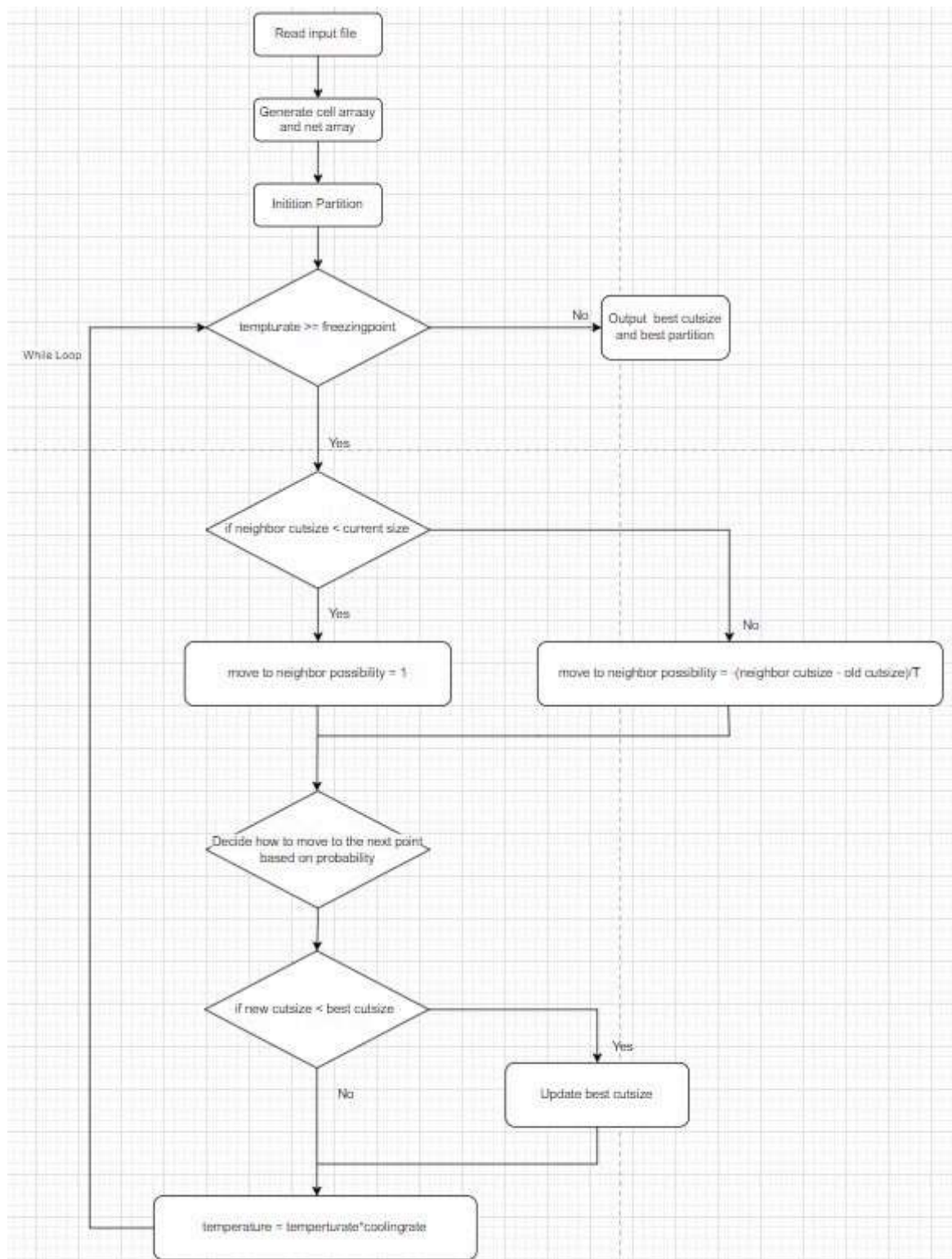
Set cooling rate cooling_rate

Set freezing point temperature freezingPoint

```
while (temperature >= freezingPoint) {
    Generate a neighbor solution new_partition
    Calculate the cost of new_partition new_cost
    Check the probability of accepting the new solution
    If accept the new solution, update current_partition and current_cost
    If new_cost is better, update best_partition and best_cost
    Decrease temperature
}
```

Generate output file, including best_partition and best_cutsizes

Flow chart



4. Program Details

Read input file to build netArray and cellArray

The input file is a netlist, so split the string and save it into a net array.

The data structure selected by cell array is **unordered_map<string, vector<string>>**, which can quickly find the net from the cell. The time complexity of finding the net from the cell is $O(1)$.

```
26 unordered_map<string, vector<string>> cellNetRelation; //cellarray
27 vector<vector<string>> myNetArray; //netarray
```

Neighbor partition data struct

```
1 struct partitionCase{
2     vector<string> groupA;
3     vector<string> groupB;
4 };
```

Initial partition

I choose to do random divisions by rolling dice for each cell under restrictive conditions.

The first one is stipulated by the question **$|\text{size}(A) - \text{size}(B)| \leq n/5$** , and the second one is set **$\text{size}(A) > \text{size}(B)$** .

```
int x = rand () % 2;
if ((x == 1 && initial.groupA.size() - initial.groupB.size() <= max_difference)
    (initial.groupA.size() <= initial.groupB.size())) {
    initial.groupA.push_back(entry.first); // push_back the cell to A
}
else {
    initial.groupB.push_back(entry.first); //push_back the cell to B
}
```

Parameter in SA algorithm

```
121 double temperature = 10;
122 double cooling_rate = 0.998;
123 double freezingPoint = 0.001;
```

Move to neighbor point

Step1. Roll the dice to determine the direction of the move and how many dice to move.

Step2. Random ordering

Step3. Move cells from source to destination

```
// Decide which direction to move and how many cells to move
do
{
    // Roll a dice randomly to decide whether to move from A to B or B to A.
    // Randomly select the number of cell(s) to be moved, set to 2
} while(//|(A - num_to_select) - (B + num_to_select)| >= (A + B) / 5);

// random ordering
random_shuffle(source.begin(), source.end());

// Move cells from source to destination
for (int i = 0; i < num_to_select; ++i) {
    destination.push_back(source.back());
    source.pop_back();
}
```

The probability of moving to the neighbor point

Calculate the probability that neighbor point moves to neighbor. If the new cutsize is better, the probability of choosing neighbor point as next time is 1. If the new cutsize is worse, the probability of selection is $\exp((\text{old_cost} - \text{new_cost}) / \text{temperature})$.

```
double move_to_neighbor_possibility(int old_cost, int new_cost, double temperat
    if (new_cost < old_cost) {          //new_cutsite better than old_cutsite
        return 1;
    } else {
        return exp((old_cost - new_cost) / temperature);
    }
}
```

Cooling Schedule

```
while(temperature >= freezingPoint) {
    // Decide how to move to neighbor point
    // Decide whether to update best cutsize, best partition
    temperature *= cooling_rate;
}
return best_partition;
```


Other modifications to the SA algorithm

As the temperature drops each time, I record not only the partitionCase in this cycle and the PartitionCase of the next neighbor, but also the PartitionCase of the previous cycle.

By calculating the probability of moving to the next neighbor point, if it is not selected, I will return to the previous PartitionCase. This design can increase the number of partitions and effectively reduce cutsize better.

```
// 查看換到neighbor的機率
if (rand() / static_cast<double>(RAND_MAX) <= move_to_neighbor_possibility(curr
    new_cost, temperature)) {
    current_partition = new_partition;
    current_cost = new_cost;
}
else{
    current_partition = previous_partition;
}
```

5. The Hardness I Encountered

How to set the parameters of SA algorithm is better?

Parameters in the SA algorithm such as initial temperature, cooling rate, and freezing point will all affect the probability of selecting neighbor point. Finally, according to the probability formula $\exp$, the selection parameters are:

```
121 double temperature = 10;
122 double cooling_rate = 0.998;
123 double freezingPoint = 0.001;
```

In SA, how many cells should be selected for movement each time will also greatly affect the results. I tried many situations during the process. Finally, it was found that in case 2, selecting two cells at a time for exchange is the best result.

num_to_select	1	2	Random 1~2	Random 1~5	Random 1~source.size()/2
Best_cutsize	240	234	241	243	258

How to improve SA by initial partition?

Many research have found that providing SA with a good initial partition at the beginning can effectively increase the efficiency of SA. Therefore, choosing other partition algorithms, such as FM and KL, can effectively reduce the final cutsize. The method I finally chose is to do SA several times more under the time constraints. This way can provide a good initial partition for the last SA.

6. My Suggestion

I think PA2 is slightly more challenging than PA1, and there are more places that need to be tried to know how to set it better. The more troublesome point is that I don't know the amount of data in other hidden cases. I don't know how big it is and whether it can be finished in ten minutes. Other than that, there is no problems. I gain a lot from this assignment.